

```
"""
ACsync Dashboard – reads results.json (produced by pipeline.py) and
displays temperature predictions, coordination decisions, and power/energy/
cost impact. Clean, minimal, light theme.
```

Run:

```
streamlit run dashboard.py
"""
```

```
import json
```

```
import pandas as pd
import plotly.graph_objects as go
import streamlit as st
```

```
st.set_page_config(page_title="ACsync", layout="wide")
```

```
# -----
# Light, minimal theme
# -----
st.markdown(
    """
    <style>
    .stApp { background-color: #FFFFFF; }
    section[data-testid="stSidebar"] { background-color: #F7F8FA; }
    h1, h2, h3 { color: #1F2937; font-weight: 600; }
    [data-testid="stMetricValue"] { color: #111827; }
    [data-testid="stMetricLabel"] { color: #6B7280; }
    .block-container { padding-top: 2rem; }
    </style>
    """,
    unsafe_allow_html=True,
)
```

```
ACCENT = "#3B82F6"      # light blue
ACCENT2 = "#F59E0B"     # light amber
GRID = "#E5E7EB"
```

```
@st.cache_data
def load_results(path="results.json"):
    with open(path, "r", encoding="utf-8") as f:
        data = json.load(f)
        temp_df = pd.DataFrame(data["temperature_data"])
```

```

    temp_df["createdAt"] = pd.to_datetime(temp_df["createdAt"])
    power_df = pd.DataFrame(data["power_data"])
    power_df["createdAt"] = pd.to_datetime(power_df["createdAt"])
    return data, temp_df, power_df

data, temp_df, power_df = load_results()

# -----
# Header
# -----
st.title("ACsync")
st.caption("Predictive Air Conditioning Coordination")
sim = data["simulation"]
st.caption(
    f"Household: {data['household']} · "
    f"Window: {sim['window_start']} → {sim['window_end']} · "
    f"{sim['rooms']} rooms"
)

# -----
# Sidebar controls
# -----
st.sidebar.header("Filters")
rooms = ["All Rooms"] + sorted(temp_df["room"].unique().tolist())
selected_room = st.sidebar.selectbox("Room", rooms)

dates = sorted(temp_df["createdAt"].dt.date.unique())
selected_date = st.sidebar.selectbox("Date", dates, format_func=str)

st.sidebar.divider()
st.sidebar.subheader("Model")
m = data["model_metrics"]
st.sidebar.write(f"***Random Forest***")
st.sidebar.write(f"RMSE: {m['random_forest_rmse_c']} °C")
st.sidebar.write(f"Trained on {m['train_houses']} houses")
st.sidebar.write(f"Tested on: {m['holdout_house']} (unseen)")
st.sidebar.write(f"Improvement vs. baseline: {m['improvement_pct']}%")

st.sidebar.divider()
st.sidebar.caption(f"Assumption: {sim['assumptions']['power_per_compressor_kw']}")
st.sidebar.caption(f"Rate: {sim['assumptions']['electricity_rate_sar_per_kwh']} $")
st.sidebar.caption(sim["assumptions"]["note"])

# -----
# System performance metrics
# -----

```

```

st.subheader("System Performance")
c1, c2, c3, c4 = st.columns(4)
c1.metric("Prediction RMSE", f"{m['random_forest_rmse_c']} °C",
          f"{m['improvement_pct']}% vs. baseline")
c2.metric("Peak Power Reduction", f"{sim['peak_reduction_pct']}%")
c3.metric("Energy Saved (7 days)", f"{sim['energy_saved_kwh']} kWh",
          f"{sim['energy_reduction_pct']}% reduction")
c4.metric("Estimated Saving", f"{sim['cost_saved_sar']} SAR")

st.divider()

# -----
# Temperature chart
# -----
st.subheader("Temperature Prediction")

day_df = temp_df[temp_df["createdAt"].dt.date == selected_date]
if selected_room != "All Rooms":
    day_df = day_df[day_df["room"] == selected_room]
else:
    day_df = day_df.groupby("createdAt", as_index=False).agg(
        temperature=("temperature", "mean"),
        predicted_temp=("predicted_temp", "mean"),
        outdoor_temp=("outdoor_temp", "mean"),
    )

fig_temp = go.Figure()
fig_temp.add_trace(go.Scatter(
    x=day_df["createdAt"], y=day_df["temperature"],
    name="Actual", line=dict(color=ACCENT, width=2)
))
if "predicted_temp" in day_df.columns:
    fig_temp.add_trace(go.Scatter(
        x=day_df["createdAt"], y=day_df["predicted_temp"],
        name="Predicted", line=dict(color=ACCENT, width=2, dash="dot")
    ))
fig_temp.add_trace(go.Scatter(
    x=day_df["createdAt"], y=day_df["outdoor_temp"],
    name="Outdoor", line=dict(color=ACCENT2, width=2)
))
fig_temp.update_layout(
    plot_bgcolor="white", paper_bgcolor="white",
    xaxis=dict(gridcolor=GRID), yaxis=dict(gridcolor=GRID, title="°C"),
    height=350, margin=dict(l=10, r=10, t=10, b=10),
    legend=dict(orientation="h", y=1.1),
)
st.plotly_chart(fig_temp, use_container_width=True)

```

```

st.divider()

# -----
# Coordination decisions table
# -----
st.subheader("ACsync Coordination")

latest_ts = day_df_room = temp_df[temp_df["createdAt"].dt.date == selected_date]
if not latest_ts.empty:
    snap_time = latest_ts["createdAt"].max()
    snapshot = temp_df[temp_df["createdAt"] == snap_time][
        ["room", "temperature", "predicted_temp", "compressor_on"]
    ].copy()
    snapshot["decision"] = snapshot["compressor_on"].map({1: "ON", 0: "OFF"})
    snapshot = snapshot.rename(columns={
        "room": "Room", "temperature": "Current (°C)",
        "predicted_temp": "Predicted (°C)", "decision": "Decision"
    })[["Room", "Current (°C)", "Predicted (°C)", "Decision"]]
    st.dataframe(snapshot, use_container_width=True, hide_index=True)

st.divider()

# -----
# Power impact chart
# -----
st.subheader("Power Impact")

power_day = power_df[power_df["createdAt"].dt.date == selected_date]
fig_power = go.Figure()
fig_power.add_trace(go.Scatter(
    x=power_day["createdAt"], y=power_day["traditional_power_kw"],
    name="Traditional", line=dict(color="#9CA3AF", width=2)
))
fig_power.add_trace(go.Scatter(
    x=power_day["createdAt"], y=power_day["acsync_power_kw"],
    name="ACsync", line=dict(color=ACCENT, width=2)
))
fig_power.update_layout(
    plot_bgcolor="white", paper_bgcolor="white",
    xaxis=dict(gridcolor=GRID), yaxis=dict(gridcolor=GRID, title="kW"),
    height=350, margin=dict(l=10, r=10, t=10, b=10),
    legend=dict(orientation="h", y=1.1),
)
st.plotly_chart(fig_power, use_container_width=True)

st.divider()

```

```

# -----
# Traditional vs ACsync summary table
# -----
st.subheader("Traditional vs. ACsync - 7-Day Summary")

summary = pd.DataFrame({
    "Metric": ["Peak power (kW)", "Average power (kW)", "Energy (kWh)", "Cost (SA
    "Traditional": [
        sim["peak_power_kw"]["traditional"], sim["avg_power_kw"]["traditional"],
        sim["energy_kwh"]["traditional"], sim["cost_sar"]["traditional"],
    ],
    "ACsync": [
        sim["peak_power_kw"]["acsync"], sim["avg_power_kw"]["acsync"],
        sim["energy_kwh"]["acsync"], sim["cost_sar"]["acsync"],
    ],
})
st.dataframe(summary, use_container_width=True, hide_index=True)

st.caption(
    f"Peak reduction: {sim['peak_reduction_pct']}% · "
    f"Energy reduction: {sim['energy_reduction_pct']}% · "
    f"Energy saved: {sim['energy_saved_kwh']} kWh"
)
st.caption(sim["assumptions"]["note"])

```